



Role

You are a senior ML researcher and technical educator who specializes in teaching self-supervised learning and vision-language architectures to software engineers transitioning into deep learning. You write with clarity, precision, and the encouraging directness of a trusted mentor.

Objective

Create a complete, structured VL-JEPA learning path document for a developer named Suhail who wants to build VL-JEPA from scratch. The document must make every foundational paper understandable before a single line of code is written.

Context

- **Learner profile:** Suhail is a software engineer with Python experience but limited ML theory background. He learns best through plain-English explanations, concrete analogies, and visual mental models — not dense math.
- **End goal:** By the end of 2 weeks, Suhail can explain every architectural component of VL-JEPA (context encoder, target encoder, predictor, language tower, cross-modal alignment) without referencing any notes.
- **Scope:** 5 core papers + 2 bonus papers, in dependency order. Each paper builds on the last. Total reading time: 6–10 hours across 2 weeks.
- **Papers to cover (in order):**
 1. *Attention Is All You Need* — Vaswani et al., 2017
 2. *An Image is Worth 16x16 Words (ViT)* — Dosovitskiy et al., 2020
 3. *CLIP: Learning Transferable Visual Models From Natural Language Supervision* — Radford et al., 2021
 4. *Masked Autoencoders Are Scalable Vision Learners (MAE)* — He et al., 2021
 5. *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA)* — Assran et al., 2023
 - Bonus 1: *DINOv2* — Oquab et al., 2023
 - Bonus 2: CLIP deep-dive (re-read Section 3.2)

Instructions

1 — Paper Entry Structure

For each of the 5 core papers, produce a section containing exactly these components in this order:

Header: Paper number, full title, authors, year, arXiv link, sections to read, estimated reading time

Why You Need This (*2–3 sentences*)

Explain how this paper connects to VL-JEPA and why skipping it would create a knowledge gap.

Plain English Summary (*the most important section*)

Cover these four points using plain language, short paragraphs, and concrete analogies:

- The problem it solved (what was broken before this paper)
- The core idea (one big insight explained simply)
- How it works step by step (numbered, no jargon unless immediately defined)
- Why it matters for VL-JEPA (explicit connection to the architecture Suhail is building)

Key Equation (*one equation maximum*)

Show the single most important equation. Define every variable in plain English on a separate line. Then provide a one-sentence intuition ("Mental model: ...").

Limitation / What Comes Next

One short paragraph explaining what this paper *cannot* do — which directly motivates the next paper in the sequence.

Mental Model Diagram

A simple ASCII or text-based flow diagram showing the paper's core pipeline (input → process → output).

2 — Comparison Table: MAE vs. I-JEPA

After Paper 4 and Paper 5 are complete, insert a side-by-side comparison table with these rows:

- What gets predicted
- Loss function
- Requires augmentations?
- What the model learns
- Main weakness

3 — Bonus Papers

For each bonus paper, provide: title, link, sections to read, estimated time, and a "Why It's Relevant" paragraph (3–5 sentences max). No full summary needed.

4 — 2-Week Reading Plan

Produce a day-by-day schedule as a markdown table with columns: Day | Paper | Task | Focus Activity.

- Week 1: Foundations (Papers 1–3)
- Week 2: Self-Supervised Learning + JEPA (Papers 4–5 + Bonus)
- Include one review/rest day per week
- Each "Focus Activity" should be a single actionable task (e.g., "Implement scaled dot-product attention in NumPy — no libraries")

5 — Key Concepts Cheat Sheet

Produce a two-column markdown table: Concept | One-Line Definition

Include at minimum these 14 terms:

Embedding, Attention, Transformer, ViT, Self-supervised learning, Contrastive learning, MAE, JEPA, EMA, Predictor, Cross-modal alignment, VL-JEPA, Zero-shot, Fine-tuning

6 — Comprehension Checklist

Produce a numbered list of 8–10

Now I have all the research needed. Here is the complete, structured VL-JEPA learning path document for Suhail.

☒ VL-JEPA Learning Path: From Zero to Architecture Fluency

A 2-Week Guided Study Program for Suhail

By: Senior ML Researcher & Technical Educator / Built specifically for software engineers entering deep learning

Your North Star: In 14 days, you will be able to whiteboard every component of VL-JEPA — context encoder, target encoder, predictor, language tower, and cross-modal alignment — without notes. Every paper below is a rung on that ladder. Skip one, and you'll hit a conceptual gap you can't climb around.

☒ The Big Picture First: What Is VL-JEPA?

Before you read a single paper, here's where you're going.

VL-JEPA (Vision-Language Joint Embedding Predictive Architecture) is a model from Meta AI, authored by DeLong Chen, Mustafa Shukor, Yann LeCun, Pascale Fung, and colleagues, submitted December 2025 and revised February 2026. Its central breakthrough: instead of teaching an AI to generate words one-by-one like a typewriter, it teaches the AI to predict the meaning of a sentence as a continuous vector in abstract space.^{[1] [2]}

Think of the difference this way:

- A classical language model says: *"Given this image, the next word is 'a', then 'dog', then 'is'..."* — it reconstructs surface text.
- VL-JEPA says: *"Given this image, the semantic embedding of what a correct description would look like is approximately here in meaning-space."* — it reasons about concepts.

The result: 50% fewer trainable parameters than token-generating VLMs with the same vision encoder and training data, native zero-shot classification, and a selective decoding mechanism that cuts inference operations by 2.85×.^[1]

To understand *why* this works, you need the five papers below. They are not optional background — they are the literal source code of VL-JEPA's design decisions.

☒ Paper 1 of 5

Paper 1 — *Attention Is All You Need*

Authors: Ashish Vaswani, Noam Shazeer, Niki Parmar et al.

Year: 2017

arXiv: arxiv.org/abs/1706.03762

Sections to Read: Abstract, Section 1 (Introduction), Section 3 (Model Architecture), Section 3.1–3.3 (Attention mechanism), Figure 1 and Figure 2.

Estimated Reading Time: 60–75 minutes

Why You Need This

VL-JEPA's context encoder, target encoder, predictor, and language tower are all Transformer-based. If you don't understand how attention works — how a model decides *which parts of an input to focus on* — you'll be building VL-JEPA blindfolded. This is the paper that introduced the Transformer architecture that now underlies virtually every modern AI model, including every component you'll implement.^{[3] [1]}

Plain English Summary

The problem it solved:

Before this paper, language models processed text sequentially — word by word — using recurrent neural networks (RNNs). Think of an RNN as a person reading a book while only holding one sentence in their head at a time. By the time they reach page 50, they've almost completely forgotten page 1. This "forgetting over distance" problem made it impossible to capture relationships between words that were far apart in a sentence or paragraph.^[4]

The core idea — one big insight:

What if, instead of reading left-to-right, every word could *simultaneously look at every other word* and decide which ones matter most? That's attention. It's like giving every word in a sentence a spotlight — and the brightness of each spotlight on every other word is learned from data. The paper called this "Scaled Dot-Product Attention," and built a whole architecture — the Transformer — entirely around it, removing recurrence completely.^[3]

How it works, step by step:

1. **Tokenize the input.** Break a sentence into tokens (subwords or words). Turn each token into a numerical vector called an embedding.
2. **Create three projections per token.** For each token embedding, compute three vectors: a Query (Q) ("What am I looking for?"), a Key (K) ("What do I offer?"), and a Value (V) ("What's my actual content?").
3. **Compute attention scores.** Multiply each token's Q vector against every other token's K vector. A high score means "these two tokens are very relevant to each other."
4. **Scale and normalize.** Divide by $\sqrt{d_k}$ (where d_k is key dimension) to prevent vanishingly small gradients. Apply softmax so scores sum to 1.
5. **Weighted sum of Values.** Multiply each score by its V vector and sum them up. Each token now holds a new representation that is a blend of all other tokens, weighted by relevance.
6. **Run multiple attention heads in parallel.** Different heads learn different types of relationships (e.g., one head tracks syntax, another tracks semantic meaning).
7. **Stack many Transformer blocks.** Each block applies attention then a feed-forward network, with residual connections and layer normalization throughout.

Why it matters for VL-JEPA:

Every encoder in VL-JEPA is a Vision Transformer (ViT), which is a direct descendant of this architecture applied to image patches instead of words. When the VL-JEPA predictor takes a context patch's encoding and tries to predict what a masked target patch's encoding *should* look like, it does so using exactly the attention mechanism described in this paper. The language tower also uses a Transformer to encode text into the embeddings VL-JEPA predicts against.^{[2] [1]}

Key Equation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q : Query matrix — what each token is "searching for" in the sequence
- K : Key matrix — what each token "advertises" about itself
- V : Value matrix — the actual information each token carries
- d_k : Dimensionality of key vectors — used for scaling to prevent tiny gradients
- $\text{softmax}(\cdot)$: Normalizes scores so they form a probability distribution summing to 1

Mental model: Think of Q as a search query in Google, K as webpage titles, and V as the actual webpage content — attention is the search engine deciding which pages to read and how much of each to blend into your answer.

Limitation / What Comes Next

Transformers were designed for 1D sequences of text tokens. An image is a 2D grid of pixels — not a sequence. If you naively flatten a 224×224 image into a sequence of pixels, you get 50,176 tokens, and attention cost scales quadratically ($O(n^2)$). That's computationally insane. The next paper solves this by treating an image as a *sequence of patches* instead.^[3]

Mental Model Diagram

INPUT TOKENS: [The] [cat] [sat] [on] [mat]

↓ ↓ ↓ ↓ ↓

Embed → Q, K, V for each token

↓

ATTENTION: Every token
looks at every other
token simultaneously

↓

New contextual representations

[The*] [cat*] [sat*] [on*] [mat*]

↓

Feed-Forward Network (per token)

↓

OUTPUT: Enriched token embeddings

Paper 2 — *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale (ViT)*

Authors: Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov et al.

Year: 2020

arXiv: arxiv.org/abs/2010.11929

Sections to Read: Abstract, Section 1, Section 3 (Method) — especially the patch embedding and position encoding parts. Figure 1 is essential.

Estimated Reading Time: 50–60 minutes

Why You Need This

VL-JEPA's context encoder and target encoder are both Vision Transformers (ViTs). The I-JEPA paper (Paper 5) is explicitly built on ViT. If you don't understand how images become patch sequences, how positional encodings work, and what the [CLS] token does, you will not understand how I-JEPA masks patches, or how VL-JEPA's target encoder processes video frames into the embeddings that the predictor tries to anticipate. ^{[2] [1]}

Plain English Summary

The problem it solved:

Before ViT, computer vision was dominated by Convolutional Neural Networks (CNNs). CNNs use sliding filters to detect local patterns like edges and textures — they have strong *inductive biases* built-in (translation invariance, locality). This makes them efficient on small datasets. But CNNs are fundamentally local — each layer only sees a neighborhood of pixels. Capturing long-range relationships (e.g., "the dog's tail position relative to its head") required stacking many layers. Transformers capture global relationships in one shot, but nobody had successfully applied them to raw images. ^{[5] [3]}

The core idea — one big insight:

Chop the image into a grid of fixed-size square patches (default: 16×16 pixels each). Flatten each patch into a vector. That's your token sequence. Feed it to a standard Transformer. The 224×224 image becomes just 196 patch-tokens — suddenly the quadratic attention cost is completely manageable. ^[3]

How it works, step by step:

1. **Patch extraction.** Divide the input image into a grid of $P \times P$ non-overlapping patches. A 224×224 image with patch size 16 gives $(224/16)^2 = 196$ patches.
2. **Linear projection (patch embedding).** Flatten each patch into a vector (16×16×3 = 768 dimensions) and multiply by a learnable weight matrix. This maps it to the transformer's hidden dimension.

3. **Prepend a [CLS] token.** Add a special learnable token at position 0. After processing, this token's output is used for classification — it aggregates information from all patches.
4. **Add positional encodings.** The Transformer has no built-in sense of order. Add a learned positional embedding to each patch so the model knows "patch 15 is in the top-right corner."
5. **Run through Transformer encoder blocks.** Standard multi-head attention + feed-forward layers, exactly as in Paper 1.
6. **Classification head.** Take the [CLS] token output, pass through a small MLP to produce class logits.

Why it matters for VL-JEPA:

I-JEPA and VL-JEPA use ViT as their encoder backbone. When VL-JEPA masks out patches and asks the predictor "what would the target encoder produce for these missing patches?", it's asking the predictor to work in ViT's patch-embedding space. The positional encodings are how VL-JEPA's predictor knows *where* the masked patches are located — it uses position tokens to tell the predictor "predict what's at grid position (3,7)" even without seeing that region.^{[1] [2]}

Key Equation

$$z_0 = [x_{\text{cls}}; x_p^1 E; x_p^2 E; \dots; x_p^N E] + E_{\text{pos}}$$

- x_p^i : i -th flattened image patch (a raw vector of pixel values)
- E : Patch embedding matrix (learnable, projects patch $\rightarrow$ hidden dimension)
- x_{cls} : Classification token (special learnable vector prepended to the sequence)
- E_{pos} : Positional embedding (adds spatial location information to each token)
- z_0 : Initial sequence fed into the Transformer (the starting representation)

Mental model: Imagine cutting a photograph into a 14×14 grid of tiles, writing the tile's grid coordinates on the back of each one, then handing the pile to a Transformer — that's exactly what this equation does.

Limitation / What Comes Next

ViT just classifies images using labels — it requires massive labeled datasets (ImageNet-21k or JFT-300M) to work well. Labels are expensive to create. What if you could train a ViT on completely unlabeled images and still get rich, useful representations? This motivates self-supervised learning — a paradigm where the model creates its own training signal from the structure of the data itself. The next paper (CLIP) is the first big leap in that direction for multimodal data.^{[6] [3]}

Mental Model Diagram

```
ORIGINAL IMAGE (224×224)
  ↓ Chop into patches
[P1][P2][P3]...[P196] ← 196 patches, each 16×16px
  ↓ Flatten + Linear Projection
[e1][e2][e3]...[e196] ← 196 patch embeddings (768-dim)
  ↓ Prepend [CLS], add position IDs
[CLS][e1+pos1][e2+pos2]...[e196+pos196]
  ↓ Transformer Encoder (12 blocks)
[CLS*][e1*][e2*]...[e196*]
  ↓ Take CLS* output only
MLP Head → Class Label
```

☒ Paper 3 of 5

Paper 3 — *Learning Transferable Visual Models From Natural Language Supervision (CLIP)*

Authors: Alec Radford, Jong Wook Kim, Chris Hallacy et al. (OpenAI)

Year: 2021

arXiv: arxiv.org/abs/2103.00020

Sections to Read: Abstract, Section 2 (Approach), Section 3.2 (Prompt engineering and ensembling), Section 4 (Experiments — skim for intuition). Focus on Figures 1 and 3.

Estimated Reading Time: 60–75 minutes

Why You Need This

CLIP introduced the idea of **cross-modal alignment** — mapping images and text into the *same* embedding space so that "a photo of a dog" and an actual photo of a dog land near each other as vectors. VL-JEPA's core innovation is to *predict in this joint embedding space* rather than decode tokens. Without understanding CLIP's contrastive alignment objective, you won't understand what VL-JEPA's language tower is trying to do or why predicting in semantic space is fundamentally different from predicting tokens.^{[7] [1]}

Plain English Summary

The problem it solved:

Classical image classifiers (like ViT trained on ImageNet) are brittle. They learn to recognize 1,000 specific categories, and if you show them a class they weren't trained on, they fail completely — even if the new class is just "a photo of a golden retriever in the snow" when they trained on "golden retriever." The model never learned *language* — it just learned label numbers. There was no bridge between visual and textual understanding.^[7]

The core idea — one big insight:

Train two separate encoders — one for images, one for text — so that matching image-text pairs

end up with similar embedding vectors, and mismatched pairs end up far apart. Use contrastive learning: in a batch of N image-text pairs, pull matching pairs together and push all $N^2 - N$ non-matching pairs apart. The training signal comes entirely from 400 million image-caption pairs scraped from the internet — no human labels needed. [\[8\]](#) [\[7\]](#)

How it works, step by step:

1. **Encode the image.** Pass an image through a ViT (or ResNet) image encoder to get a fixed-size embedding vector v_i .
2. **Encode the text.** Pass the corresponding caption through a Transformer text encoder to get a text embedding vector t_i .
3. **Project to shared space.** Apply a linear projection to both v_i and t_i to map them into a shared dimension. L2-normalize both.
4. **Compute similarity matrix.** For a batch of N pairs, compute the $N \times N$ cosine similarity matrix between all image embeddings and all text embeddings.
5. **Apply contrastive loss.** The diagonal of that matrix (true pairs) should have high similarity. Everything off-diagonal (wrong pairs) should have low similarity. The InfoNCE loss penalizes deviations from this.
6. **Zero-shot inference.** At test time, to classify an image, encode phrases like "a photo of a cat" and "a photo of a dog" into text embeddings, then find which text embedding is most similar to the image embedding. No fine-tuning needed.

Why it matters for VL-JEPA:

VL-JEPA's language tower encodes text descriptions into embeddings — and VL-JEPA's predictor is trained to predict *those embeddings* from video context. This is a direct extension of CLIP's insight: if you can align vision and language in a shared space (CLIP's contribution), you can then build a predictive architecture that operates entirely inside that shared space (VL-JEPA's contribution). VL-JEPA surpasses CLIP on 8 video classification and 8 video retrieval benchmarks precisely because it doesn't just *align* these spaces — it learns to *predict* across them. [\[1\]](#)

Key Equation

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(v_i, t_i)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(v_i, t_j)/\tau)}$$

- v_i : Image embedding for the i -th sample (output of image encoder + projection)
- t_i : Text embedding for the i -th sample (output of text encoder + projection)
- $\text{sim}(\cdot, \cdot)$: Cosine similarity — measures directional closeness of two vectors
- τ : Temperature parameter — controls how "peaky" the distribution is (learned)
- N : Batch size — all other text embeddings in the batch are treated as negatives

Mental model: Imagine a room full of people holding name tags (text) and photos (images). CLIP's loss says: "The person in *this* photo should end up standing right next to the person

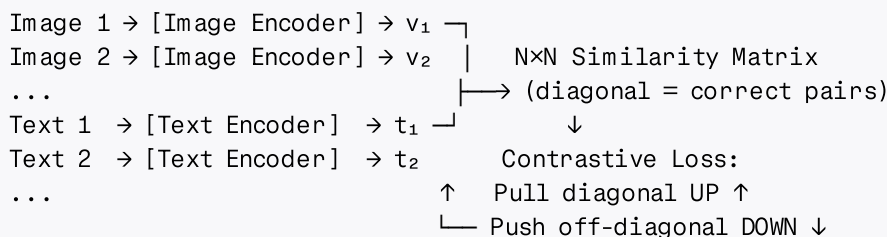
holding *this* name tag — and as far as possible from everyone else."

Limitation / What Comes Next

CLIP learns shared representations, but it requires **massive amounts of negative examples** (large batch sizes) and **carefully crafted contrastive pairs** to work. It also learns at the image level — not at the patch level. It doesn't ask: "What is happening in *this specific region* of the image?" There's also no mechanism for the model to reason about what's *missing* from an image or to predict unobserved content. That gap — learning rich local representations *without* contrastive pairs — is exactly what MAE (the next paper) addresses.^[7]

Mental Model Diagram

BATCH of N image-text pairs:



RESULT: Shared embedding space where
"photo of dog" $\approx$ [dog image vector]

☒ Paper 4 of 5

Paper 4 — *Masked Autoencoders Are Scalable Vision Learners (MAE)*

Authors: Kaiming He, Xinlong Wang, Saining Xie, Ross Girshick et al. (Meta AI)

Year: 2021

arXiv: arxiv.org/abs/2111.06377

Sections to Read: Abstract, Section 1, Section 2 (Approach — all subsections), Section 4.1 (Main Properties), Figure 1 and Figure 3.

Estimated Reading Time: 50–60 minutes

Why You Need This

MAE is the direct architectural ancestor of I-JEPA and VL-JEPA's masking philosophy. It proved that you can train ViT to produce powerful representations by masking out 75% of image patches and training the model to reconstruct the missing pixel values. VL-JEPA keeps the masking insight but fundamentally changes *what* gets predicted — not pixels, but semantic embeddings. Understanding MAE's architecture and its critical weakness (pixel-space prediction) is the exact motivation for switching to JEPA-style prediction.^{[9] [10]}

Plain English Summary

The problem it solved:

ViT works great for images, but it needs millions of labeled examples to train from scratch — expensive and slow. Self-supervised learning (SSL) for NLP had already solved this problem: BERT masks out words and trains the model to predict them. Why couldn't you do the same for images? Early attempts failed because images are *spatially redundant* — neighboring pixels are nearly identical, so it's too easy for the model to "cheat" by interpolating from adjacent unmasked pixels. The model learns nothing deep.^[9]

The core idea — one big insight:

Mask out a *very high fraction* of patches — 75%, not 15–25% like BERT. At this extreme masking ratio, simple interpolation becomes impossible. The model is forced to build a semantic understanding of the whole image to reconstruct the invisible regions. Additionally, only the *visible* patches (25%) are fed to the encoder — the decoder sees masked patches as learnable tokens. This asymmetric design makes MAE extremely fast to pretrain.^{[10] [9]}

How it works, step by step:

1. **Patch and embed.** Split image into patches, embed each one (same as ViT).
2. **Random masking.** Randomly remove 75% of patches from the sequence. Only the visible 25% are passed to the encoder.
3. **Encoder processing.** Run the full ViT encoder on the visible patches only. This is the expensive part — but it only processes 25% of patches, so it's very fast.
4. **Decoder input assembly.** Insert learnable mask tokens (shared, learned vectors representing "unknown patch") at all masked positions, with positional encodings so the decoder knows where each missing patch belongs.
5. **Decoder reconstruction.** A smaller, lightweight decoder (not the same as the encoder) processes the full sequence and predicts the pixel values of each masked patch.
6. **Loss on masked patches only.** Compute mean squared error (MSE) between predicted pixels and actual pixels *only for the masked patches*. Visible patches don't contribute to the loss.
7. **Discard decoder at inference.** After pretraining, throw away the decoder. Keep only the encoder to produce powerful patch-level representations for downstream tasks.

Why it matters for VL-JEPA:

MAE demonstrates that aggressive masking forces models to learn semantic representations — this exact intuition lives inside I-JEPA and VL-JEPA. But MAE's critical flaw is that it predicts raw pixel values. Pixel prediction is not semantic — you can perfectly reconstruct pixel colors without understanding anything about the *meaning* of what's in the image. VL-JEPA completely abandons pixel-space prediction in favor of embedding-space prediction, where representations encode *meaning* rather than appearance.^{[9] [1]}

Key Equation

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|^2$$

- $\mathcal{M}$: Set of masked patch indices (the 75% that were hidden)
- $|\mathcal{M}|$: Number of masked patches (normalizes the loss)
- x_i : True pixel values of the i-th masked patch (the ground truth)
- $\hat{x}_i$: Predicted pixel values for the i-th masked patch (decoder output)
- $\|\cdot\|^2$: Squared L2 norm — measures how far the prediction is from the truth

Mental model: You cover 75% of a photograph with sticky notes, then ask someone to draw what's behind each sticky note. You grade them purely on whether the color values they drew match the original photo exactly — but you don't care if they *understand* what's in the image at all.

Limitation / What Comes Next

MAE predicts in **pixel space** — it reconstructs exact RGB color values. This means the model is rewarded for getting pixel-accurate textures right, not for understanding *what objects are*. A model that perfectly reconstructs blurry grass still scores well on MAE's loss. There's no guarantee that the representations are semantically meaningful. The next paper (I-JEPA) fixes this by predicting in **representation space** — it asks the model to predict *semantic embeddings*, not pixels.^{[10] [9]}

Mental Model Diagram

```
FULL IMAGE (196 patches)
  ↓ Mask 75% randomly
[v][v][ ][ ][v][ ][ ][v]... (v=visible, blank=masked)
  ↓ Encoder sees only visible patches
[ViT Encoder] ← runs on 25% of patches only
  ↓ Produces rich embeddings for visible patches
[visible embeddings] + [mask tokens at masked positions]
  ↓ Decoder (smaller, lightweight)
[ViT Decoder] ← processes ALL positions
  ↓
Predicted pixel values for all 196 patches
  ↓
Loss: MSE on masked patches only
```

Paper 5 — *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA)*

Authors: Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, Nicolas Ballas (Meta AI)

Year: 2023

arXiv: arxiv.org/abs/2301.08243

Sections to Read: Abstract, Section 1, Section 2 (Method) — all subsections, Section 3.1 (Image Representation Evaluation), Figure 1 (architecture), Figure 2 (masking strategy).

Estimated Reading Time: 75–90 minutes

Why You Need This

I-JEPA is the direct architectural blueprint for VL-JEPA. VL-JEPA is, in essence, I-JEPA extended to video and language. Every component in VL-JEPA — context encoder, target encoder (with EMA), predictor with positional tokens — is inherited from I-JEPA. If you understand I-JEPA completely, VL-JEPA becomes a set of deliberate, logical extensions rather than a collection of opaque design choices.^{[11] [1]}

Plain English Summary

The problem it solved:

MAE (Paper 4) predicts pixels — and that's too low-level. DINO and other contrastive SSL methods predict in embedding space but require aggressive data augmentation (flipping, cropping, color jittering) to generate positive pairs — and those augmentations bake in biases about *what should be invariant*. I-JEPA asks: can we learn highly semantic representations without pixel prediction and without any data augmentation at all — just by predicting context-to-context inside a single image?^{[12] [11]}

The core idea — one big insight:

Take one image, no augmentations. Mask out several target regions. Give the model a visible context region. Train it to predict *what the target encoder would output* for each masked region — not the pixels, but the semantic representation. The critical trick: the **target encoder's weights are never directly trained** — they are an exponential moving average (EMA) of the context encoder's weights. This prevents a collapse where both encoders output the same constant vector regardless of input.^{[13] [12]}

How it works, step by step:

1. Patch the image. Split image into N non-overlapping patches.
2. Run target encoder on the whole image. The target encoder (weights = EMA of context encoder) processes *all* N patches and produces N target representations $\{s_{y_1}, \dots, s_{y_N}\}$.

3. **Sample target blocks.** Randomly sample M target blocks — contiguous rectangular regions of patches. These are the representations the model must predict.
4. **Sample context block.** Sample one larger context block. Any overlap with target blocks is masked out from the context, so the model can't cheat by seeing the target directly.
5. **Run context encoder on visible context.** The context encoder processes only the unmasked context patches.
6. **Predictor with position tokens.** A smaller Transformer predictor takes the context encoder's output plus positional encodings for each target location. It asks: "Given what I see in the context, what would the target encoder output at position (row, col)?"
7. **L2 loss between prediction and target.** The predictor's output for each target patch is compared to the target encoder's actual representation for that patch. Loss = average L2 distance.
8. **Update rules.** Context encoder and predictor weights are updated by gradient descent. Target encoder weights are **never updated by gradients** — only by EMA:

$$\theta_{\text{target}} \leftarrow \alpha \theta_{\text{target}} + (1 - \alpha) \theta_{\text{context}}.$$

Why it matters for VL-JEPA:

VL-JEPA is I-JEPA with three additions: (1) it operates on video (spatiotemporal patches), (2) it adds a language tower that encodes text into the same space, and (3) instead of predicting visual patch representations, the predictor predicts *text embeddings*. The context/target encoder duo and the EMA trick are carried forward verbatim into VL-JEPA's video encoder architecture. [2] [1]

Key Equation

$$\mathcal{L} = \frac{1}{M} \sum_{j=1}^M \|\hat{s}_{y_j} - s_{y_j}\|_2^2$$

- M : Number of target blocks sampled in this training step
- $\hat{s}_{y_j}$: Predicted representation for target block j (output of the predictor)
- s_{y_j} : Target representation for block j (output of the EMA target encoder — the ground truth)
- $\|\cdot\|_2^2$: Squared L2 distance — how far the prediction is from the target in embedding space

Mental model: Imagine you can only see the left half of a painting. Your task isn't to paint the right half in pixel-perfect detail — it's to write a semantic description that matches what an expert art critic would write about the right half. You're judged on conceptual accuracy, not color matching.

Limitation / What Comes Next

I-JEPA only works on static images with one modality. There's no language. There's no temporal understanding of how the world changes over time. You can't ask "what's happening in this video?" or "does this scene match this description?" VL-JEPA takes everything I-JEPA built and extends it: first to spatiotemporal video patches (as V-JEPA did), then adds a language tower so the predictor operates in a joint vision-language embedding space. ^[12] ^[1]

Mental Model Diagram

```
SINGLE IMAGE (no augmentations)
  ↓ Split into patches
Target Encoder (EMA – no gradients)
  ↓ Process ALL patches
[s_y1][s_y2]...[s_yN] ← target representations

    Randomly sample:
    - M target blocks (what to predict)
    - 1 context block (what to learn from)

Context Encoder (trained by gradients)
  ↓ Process visible context patches only
[context embeddings]
  ↓
PREDICTOR (small Transformer)
Input: context embeddings + position tokens for targets
  ↓
[ŝ_y1][ŝ_y2]...[ŝ_yM] ← predicted representations

  ↓
L2 Loss: predicted vs. target encoder's output
(No pixel reconstruction. No augmentations.)
```

☒ MAE vs. I-JEPA: Side-by-Side Comparison

Dimension	MAE (He et al., 2021)	I-JEPA (Assran et al., 2023)
What gets predicted	Raw pixel values of masked patches ^[9]	Semantic embeddings produced by the EMA target encoder ^[12]
Loss function	MSE in pixel space on masked patches only ^[9]	L2 distance in representation space between predictor output and target encoder output ^[11]
Requires augmentations?	No — masking is the only perturbation ^[10]	No — masking only, explicitly designed to avoid augmentation bias ^[11]
What the model learns	Low-to-mid level features; good for texture and local patterns ^[9]	High-level semantic representations; better for object counting, depth prediction, scene understanding ^[12]

Dimension	MAE (He et al., 2021)	I-JEPA (Assran et al., 2023)
Main weakness	Predicting pixels $\neq$ understanding meaning; models can score well without semantic understanding ^[10]	Requires careful tuning of target block scale and context informativeness; collapses without EMA target encoder ^[12]

☒ Bonus Papers

Bonus 1 — *DINOv2: Learning Robust Visual Features without Supervision*

Authors: Maxime Oquab, Timothée Darcet, Théo Moutakanni et al. (Meta AI)

Year: 2023

arXiv: arxiv.org/abs/2304.07193

Sections to Read: Abstract, Section 2 (Method overview), Section 3.1 (Data pipeline), Section 4 (results tables — skim).

Estimated Time: 45–60 minutes

Why It's Relevant:

DINOv2 is a self-supervised ViT that produces some of the best visual representations available without any labels. It uses a self-distillation approach (like I-JEPA's EMA trick, but with an online/offline teacher-student pair) on curated web-scale data. VL-JEPA's vision encoder can be initialized with DINOv2-style pretraining before the language alignment stage, and Théo Moutakanni, one of DINOv2's co-authors, is also a co-author of VL-JEPA. Understanding DINOv2 shows you why strong visual foundation models are the prerequisite for any effective vision-language architecture. It also introduces the concept of "patch-level features" that are so semantically rich they can do segmentation without fine-tuning — a direct precedent for VL-JEPA's native open-vocabulary classification ability.^[1]

Bonus 2 — *CLIP Deep-Dive: Re-read Section 3.2 (Prompt Engineering and Ensembling)*

Source: Original CLIP paper — arxiv.org/abs/2103.00020, Section 3.2 only.

Estimated Time: 20–30 minutes

Why It's Relevant:

VL-JEPA's selective decoding mechanism — which invokes a lightweight text decoder "only when needed" and reduces decoding operations by 2.85× — requires the model to work effectively in embedding space without always converting back to text tokens. Section 3.2 of CLIP explains *prompt engineering*: how the way you construct your text queries fundamentally affects embedding quality and zero-shot accuracy. When VL-JEPA performs open-vocabulary classification (no architecture modification required), it is essentially doing CLIP-style embedding comparison at inference — understanding how CLIP handles prompt sensitivity tells you exactly what failure modes to expect in VL-JEPA's zero-shot classification head and how to mitigate them in your implementation.^{[7] [1]}

☒ 2-Week Reading Plan

Day	Paper	Task	Focus Activity
Mon — Day 1	Paper 1: <i>Attention Is All You Need</i>	Read Abstract + Sections 1, 3.1, 3.2, and Figure 1	Draw the full attention mechanism from memory on paper: $Q, K, V \rightarrow \text{scores} \rightarrow \text{softmax} \rightarrow \text{weighted sum}$
Tue — Day 2	Paper 1: <i>Attention Is All You Need</i>	Re-read Section 3 fully. Implement scaled dot-product attention.	Implement scaled dot-product attention in NumPy — no PyTorch, no libraries: $Q @ K.T / \sqrt{d_k}$
Wed — Day 3	Paper 2: <i>ViT</i>	Read Abstract + Section 1 + Section 3 (Method) + Figure 1	Write code to convert a 224×224 image into a sequence of 196 flattened 16×16 patches using only NumPy
Thu — Day 4	Paper 2: <i>ViT</i>	Re-read Section 3 + study positional encodings	Implement a toy ViT patch-embedding layer in PyTorch: image $\rightarrow$ patches $\rightarrow$ linear projection $\rightarrow$ add position embeddings
Fri — Day 5	Paper 3: <i>CLIP</i>	Read Abstract + Section 2 (Approach) + Figure 1	Implement contrastive loss (InfoNCE) in PyTorch for a toy batch of 4 image-text pairs using random embeddings
Sat — Day 6	Paper 3: <i>CLIP</i>	Read Section 3.2 (Prompt Engineering) + Section 4 (skim results)	Design 5 different text prompts for the same image concept; predict which would have highest cosine similarity — then verify with a CLIP demo
Sun — Day 7	☒ Review Day	Consolidate Week 1 — no new reading	Write a 1-page "explain it to a 10-year-old" summary of Attention, ViT, and CLIP; then draw all three architectures from memory
Mon — Day 8	Paper 4: <i>MAE</i>	Read Abstract + Section 1 + Section 2 (all subsections) + Figure 1	Implement the MAE masking strategy: write a function that takes 196 patch indices and randomly samples 75% to mask
Tue — Day 9	Paper 4: <i>MAE</i>	Re-read Section 2 + Section 4.1 (Main Properties)	Implement the asymmetric encoder-decoder in PyTorch: encoder on visible patches only; decoder reconstructs all positions
Wed — Day 10	Paper 5: <i>HJEPa</i>	Read Abstract + Section 1 + Section 2 (Method) + Figures 1–2	Write pseudocode for the I-JEPa training loop: label every step (mask sampling $\rightarrow$ context encoder $\rightarrow$ predictor $\rightarrow$ EMA update)
Thu — Day 11	Paper 5: <i>HJEPa</i>	Re-read Section 2 + Section 3.1	Implement the EMA update rule from scratch: $\text{target} = \alpha * \text{target} + (1 - \alpha) * \text{context}$ — test that weights diverge without it

Day	Paper	Task	Focus Activity
Fri — Day 12	Bonus 1: <i>DINOv2</i>	Read Abstract + Section 2 + Section 3.1	Compare DINOv2's teacher-student setup to I-JEPA's EMA design in a written table: 5 similarities, 5 differences
Sat — Day 13	Bonus 2: <i>CLIP Section 3.2 re-read</i> + VL-JEPA paper: arxiv.org/abs/2512.10942	Read VL-JEPA Abstract + Introduction + Architecture section	Map each VL-JEPA component to the paper it came from: draw a dependency tree (context encoder ← I-JEPA; language tower ← CLIP; etc.)
Sun — Day 14	☒ Final Review Day	No new reading	Whiteboard drill: explain every VL-JEPA component in under 2 minutes each, out loud, without notes. Record yourself on your phone.

☒ Key Concepts Cheat Sheet

Concept	One-Line Definition
Embedding	A vector of numbers that represents an object (word, image patch, video clip) in a mathematical space where similar things are geometrically close ^[12]
Attention	A mechanism where every element in a sequence learns to weight how much it should "pay attention" to every other element, computed via queries, keys, and values ^[3]
Transformer	A neural network architecture built entirely from stacked attention layers and feed-forward blocks, with no recurrence or convolution ^[3]
ViT	Vision Transformer — a Transformer that processes images by splitting them into fixed-size patches and treating each patch as a token in a sequence ^[3]
Self-supervised learning	Training paradigm where the model generates its own supervision signal from unlabeled data (e.g., by predicting masked parts of an input), requiring no human annotations ^[12]
Contrastive learning	An SSL approach that trains a model by pulling embeddings of matching pairs together and pushing embeddings of non-matching pairs apart in representation space ^[7]
MAE	Masked Autoencoder — an SSL method that masks 75% of image patches and trains a model to reconstruct the original pixel values of hidden patches ^[9]
JEPA	Joint Embedding Predictive Architecture — a framework where a model predicts the <i>representation</i> of a target from a context, operating in semantic embedding space rather than pixel or token space ^[12]
EMA	Exponential Moving Average — a technique to update the target encoder's weights as a slow-moving average of the context encoder's weights, preventing representation collapse without contrastive negatives ^[12]
Predictor	In JEPA architectures, a lightweight Transformer network that takes context encoder output and positional tokens, then outputs predicted embeddings for masked target locations ^[12]
Cross-modal alignment	The process of training encoders from different modalities (e.g., images and text) so that semantically matching inputs (a photo and its caption) produce similar embedding vectors ^[7]
VL-JEPA	Vision-Language JEPA — a Meta AI model that predicts continuous text embeddings (not tokens) from video context, achieving strong multimodal understanding with 50% fewer parameters than classical VLMs ^[1]

Concept	One-Line Definition
Zero-shot	Performing a task (e.g., classifying an image into a new category) without any task-specific training examples, by leveraging the general structure of a pretrained embedding space [8]
Fine-tuning	Taking a pretrained model and continuing training on a smaller task-specific dataset to adapt its representations to a downstream objective [6]

✓ Comprehension Checklist

You are ready to build VL-JEPA when you can answer all of the following from memory — no notes, no papers:

1. Explain the attention mechanism using only an analogy. Describe, in plain English, what Q, K, and V represent without using the words "query," "key," or "value" — use a real-world analogy instead.
2. Trace a single image patch through a ViT encoder. Starting from raw 16×16 pixel values, describe every transformation — linear projection, positional encoding, multi-head attention, feed-forward, layer norm — until you have a final patch embedding.
3. Explain why CLIP doesn't need labeled data. Describe what a "positive pair" and "negative pair" are in CLIP, and why the internet-scraped alt-text of images is sufficient supervision for alignment.
4. State the single most important difference between MAE and I-JEPA. Describe the specific change in *what gets predicted* and explain why that change produces more semantically meaningful representations.
5. Explain the EMA trick without equations. Why can't both the context encoder and target encoder be updated by gradient descent? What goes wrong if they are, and how does EMA prevent it?
6. Describe the I-JEPA masking strategy and why it matters. Why does I-JEPA sample *large* target blocks and an *informative context* block? What happens to representation quality if target blocks are too small?
7. Map every VL-JEPA component to its origin paper. For each of: context encoder, target encoder, predictor, language tower, and cross-modal alignment objective — name the paper that introduced that idea and what VL-JEPA adds or changes.
8. Explain VL-JEPA's "selective decoding" in plain English. What is the text decoder doing, when is it invoked, and why does predicting in embedding space allow you to skip it 2.85× more often than token-space generation?
9. Contrast CLIP-style alignment with VL-JEPA's prediction objective. CLIP pulls matching image-text embeddings together; VL-JEPA predicts the text embedding from video context. What is the fundamental difference in what each model *learns to do* during training?
10. State the two failure modes a beginner is most likely to hit when implementing I-JEPA from scratch. (Hint: one involves the target encoder. One involves the masking ratio and block size.) Describe what each failure looks like in the loss curve and how you would diagnose it.

Suhail, you've got this. These 5 papers cover roughly 8 years of architectural evolution — from the 2017 Transformer to the December 2025 VL-JEPA. Each one is a direct stepping stone. Follow the schedule, do the coding exercises (they matter more than re-reading), and use the comprehension checklist as your daily self-test. When you can answer all 10 questions cold, you're not just ready to build VL-JEPA — you're ready to extend it.

✱

1. <https://arxiv.org/abs/2512.10942>
2. <https://arxiv.org/pdf/2512.10942.pdf>
3. <https://theaisummer.com/vision-transformer/>
4. https://www.reddit.com/r/MachineLearning/comments/vtsiif/d_paper_explained_jepa_a_path_towards_autonomous/
5. <https://www.v7labs.com/blog/vision-transformer-guide>
6. <https://towardsdatascience.com/vision-transformers-vit-explained-are-they-better-than-cnns/>
7. <https://viso.ai/deep-learning/clip-machine-learning/>
8. <https://openai.com/index/clip/>
9. <https://arxiv.org/abs/2111.06377>
10. <https://mchromiak.github.io/articles/2021/Nov/14/Masked-Autoencoders-Are-Scalable-Vision-Learners/>
11. https://openaccess.thecvf.com/content/CVPR2023/papers/Assran_Self-Supervised_Learning_From_Images_With_a_Joint-Embedding_Predictive_Architecture_CVPR_2023_paper.pdf
12. <https://encord.com/blog/i-jepa-explained/>
13. <https://debuggercafe.com/jepa-series-part-1-introduction-to-i-jepa/>